

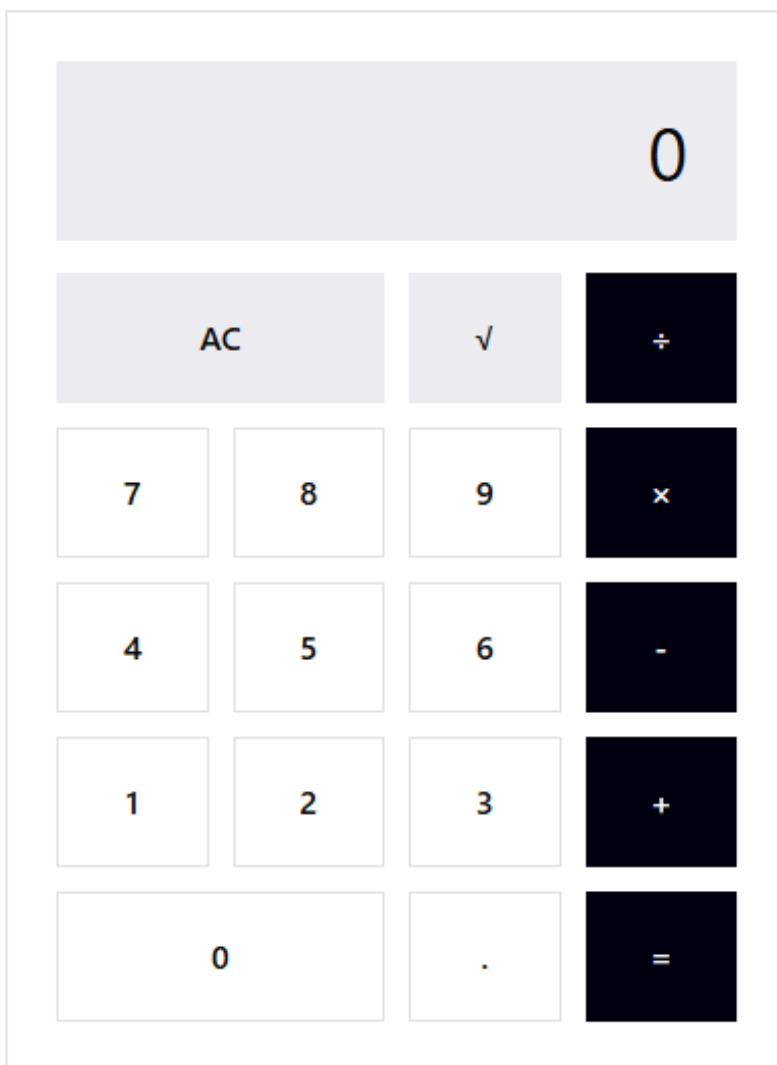
# Projeto - Calculadora

## Requisitos

Esse projeto tem como objetivo desenvolver uma calculadora simples com React Native e Expo, para realizar operações matemáticas básicas. Sendo assim, permite ao usuário efetuar soma, subtração, multiplicação, divisão e raiz quadrada, em uma interface contendo botões numéricos e das operações mencionadas.

## Protótipo

Imagem do protótipo realizado no Figma.



Documento do Negócio: Print da Tela do Sistema em Funcionamento

|           |   |   |   |
|-----------|---|---|---|
| 1589+3687 |   |   |   |
| AC        | √ | / |   |
| 7         | 8 | 9 | * |
| 4         | 5 | 6 | - |
| 1         | 2 | 3 | + |
| 0         | . | = |   |

|      |   |   |   |
|------|---|---|---|
| 5276 |   |   |   |
| AC   | √ | / |   |
| 7    | 8 | 9 | * |
| 4    | 5 | 6 | - |
| 1    | 2 | 3 | + |
| 0    | . | = |   |

## Requisitos

RF001: Deve permitir realizar operações matemáticas com até 2 argumentos, retornando 1 informação.

RF002: Deve conter visor para exibição dos valores informados pelo usuário.

RF003: Deve realizar as operações de soma, subtração, multiplicação, divisão e raiz quadrada.

RF004: Deve exibir os resultados na tela diante dos valores digitados e calcular ao pressionar o botão de igual “=”.

RF005: Deve possuir botões para os números (0 a 9) e para as operações mencionadas.

RF006: Deve permitir limpar os valores inseridos na tela.

RF007: Deve impedir que operadores sejam duplicados em sequência. Exemplo “5 + + 5”.

RNF001: Aplicação deve ser desenvolvida com React Native + Expo.

## Código

```
import { useState } from "react";
import { StyleSheet, Text, TouchableOpacity, View } from
"react-native";

export default function App() {
  const [display, setDisplay] = useState("0");

  const handleOperator = (operator: string) => {
    const lastChar = display[display.length - 1];

    if (lastChar !== "+" && lastChar !== "-" && lastChar !== "*" &&
lastChar !== "/") {
      setDisplay(display + operator);
    }
  };

  const handleSqrt = () => {
    const num = parseInt(display);
    const resultado = Math.sqrt(num);
    setDisplay(resultado.toString());
  };
};
```

```
const handleEquals = () => {
  if (display.includes("+")) {
    const partes = display.split("+");
    const num1 = parseInt(partes[0]);
    const num2 = parseInt(partes[1]);
    const resultado = num1 + num2;
    setDisplay(resultado.toString());
  }

  else if (display.includes("-")) {
    const partes = display.split("-");
    const num1 = parseInt(partes[0]);
    const num2 = parseInt(partes[1]);
    const resultado = num1 - num2;
    setDisplay(resultado.toString());
  }

  else if (display.includes("*")) {
    const partes = display.split("*");
    const num1 = parseInt(partes[0]);
    const num2 = parseInt(partes[1]);
    const resultado = num1 * num2;
    setDisplay(resultado.toString());
  }

  else if (display.includes("/")) {
    const partes = display.split("/");
    const num1 = parseInt(partes[0]);
    const num2 = parseInt(partes[1]);
    const resultado = num1 / num2;
    setDisplay(resultado.toString());
  }
};

return (
  <View style={styles.container}>

    <View style={styles.displayBox}>
      <Text style={styles.display}>{display}</Text>
    </View>
  </View>
);
```

```

</View>

<View style={styles.row}>

  <TouchableOpacity style={[styles.button, styles.largeButton]}
onPress={() => setDisplay("0")}>
    <Text style={styles.buttonText}>AC</Text>
  </TouchableOpacity>

  <TouchableOpacity style={styles.button} onPress={handleSqrt}>
    <Text style={styles.buttonText}>√</Text>
  </TouchableOpacity>

  <TouchableOpacity style={styles.button} onPress={() =>
handleOperator("/")}>
    <Text style={styles.buttonText}>/</Text>
  </TouchableOpacity>

</View>

<View style={styles.row}>

  <TouchableOpacity style={styles.button} onPress={() =>
setDisplay(display === "0" ? "7" : display + "7")}>
    <Text style={styles.buttonText}>7</Text>
  </TouchableOpacity>

  <TouchableOpacity style={styles.button} onPress={() =>
setDisplay(display === "0" ? "8" : display + "8")}>
    <Text style={styles.buttonText}>8</Text>
  </TouchableOpacity>

  <TouchableOpacity style={styles.button} onPress={() =>
setDisplay(display === "0" ? "9" : display + "9")}>
    <Text style={styles.buttonText}>9</Text>
  </TouchableOpacity>

  <TouchableOpacity style={styles.button} onPress={() =>
handleOperator("*")}>
    <Text style={styles.buttonText}>*</Text>
  </TouchableOpacity>

</View>

```

```
<View style={styles.row}>

  <TouchableOpacity style={styles.button} onPress={() =>
setDisplay(display === "0" ? "4" : display + "4")}>
    <Text style={styles.buttonText}>4</Text>
  </TouchableOpacity>

  <TouchableOpacity style={styles.button} onPress={() =>
setDisplay(display === "0" ? "5" : display + "5")}>
    <Text style={styles.buttonText}>5</Text>
  </TouchableOpacity>

  <TouchableOpacity style={styles.button} onPress={() =>
setDisplay(display === "0" ? "6" : display + "6")}>
    <Text style={styles.buttonText}>6</Text>
  </TouchableOpacity>

  <TouchableOpacity style={styles.button} onPress={() =>
handleOperator("-")}>
    <Text style={styles.buttonText}>-</Text>
  </TouchableOpacity>

</View>

<View style={styles.row}>

  <TouchableOpacity style={styles.button} onPress={() =>
setDisplay(display === "0" ? "1" : display + "1")}>
    <Text style={styles.buttonText}>1</Text>
  </TouchableOpacity>

  <TouchableOpacity style={styles.button} onPress={() =>
setDisplay(display === "0" ? "2" : display + "2")}>
    <Text style={styles.buttonText}>2</Text>
  </TouchableOpacity>

  <TouchableOpacity style={styles.button} onPress={() =>
setDisplay(display === "0" ? "3" : display + "3")}>
    <Text style={styles.buttonText}>3</Text>
  </TouchableOpacity>
```

```

        <TouchableOpacity style={styles.button} onPress={() =>
handleOperator("+")}>
            <Text style={styles.buttonText}>+</Text>
        </TouchableOpacity>

    </View>

    <View style={styles.row}>

        <TouchableOpacity style={[styles.button, styles.largeButton]}
onPress={() => setDisplay(display !== "0" ? display + "0" : "0")}>
            <Text style={styles.buttonText}>0</Text>
        </TouchableOpacity>

        <TouchableOpacity style={styles.button} onPress={() =>
setDisplay(display + ".")}>
            <Text style={styles.buttonText}>.</Text>
        </TouchableOpacity>

        <TouchableOpacity style={styles.button} onPress={handleEquals}>
            <Text style={styles.buttonText}>=</Text>
        </TouchableOpacity>

    </View>

</View>

);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#b5d5f6", paddingTop: 40,
alignItems: "center" },
  displayBox: {width: 320,height: 80, backgroundColor: "#fdffff",
borderWidth: 1,
  justifyContent: "center",paddingHorizontal: 15},
  display: { fontSize: 35, textAlign: "right" },
  row: { flexDirection: "row", justifyContent: "center", gap: 10,
marginVertical: 8 },
  button: {
    borderWidth: 1,
    paddingVertical: 12,

```

```
paddingHorizontal: 20,  
width: 70,  
alignItems: "center",  
justifyContent: "center",  
backgroundColor: "#fff"  
},  
  
largeButton: { width: 140, marginHorizontal: 10 },  
buttonText: { fontSize: 25 }  
});
```